**Code Jam 2022 - World Finals**

# Analysis: Goose, Goose, Ducks?

## Test Set 1

Consistency in this problem has a property which is not true for most typical logic systems, in that a set of statements is consistent if and only if each subset of size $2$ of it is consistent. The left to right implication is trivial, but the converse is not, so let us prove it.

Assume you have a set of statements $S$ such that any two of them are consistent. Then, for each bird $b$, consider all statements that state that $b$ is at a specific point in time and space. If we sort all those points by time and linearly interpolate between consecutive points, we obtain a path for $b$ that is consistent with all statements in $S$. Notice that, because any pair of statements is consistent — in particular, pairs of consecutive statements — the linear interpolation part does not exceed the maximum speed. In this way, we can obtain a path for each mentioned bird, all of which are consistent with all statements. Then, by definition, $S$ is consistent.

Meetings also state "this bird was at this time-space point" for birds that are ducks. Thus, an analogous reasoning shows that a set of statements and known ducks is consistent with the meetings if and only if they are pairwise consistent. Since the meetings are pairwise consistent according to the limits, this leaves only pairs of two statements and pairs of a statement a meeting to check.

At this point, with the limits in Test Set 1 being so small, we can simply try everything. We know there is at least one duck, so start by trying every bird as "the first duck". Within each option, go through statements and check them against meetings (if they involve a known duck) and against previous statements. If a statement contradicts a meeting, the issuer of that statement must be a duck. If a statement contradicts a previous statement, then the issuer of such previous statement must be a duck (since ducks cannot contradict geese). Each time we find a new duck, we start checking everything again. When we go through all statements without finding any contradictions with our current set of ducks, we are done and we have a candidate set of ducks. Notice that all birds being ducks is always a valid answer. Finally, we keep the smallest set of ducks and output its size.

This solution requires $\mathbf{N}$ iterations of the outer loop, to try every possible "first duck". Checking a pair of statements or a statement and a meeting for consistency takes constant time, as it is only checking whether birds that are mentioned in both can get from one point in space-time to another, which is a simple bit of math. Thus, checking every statement against every other statement and every meeting takes $O(\mathbf{S} \times (\mathbf{S} + \mathbf{M}))$. On every iteration through statements except for one (the last one) we find at least one additional duck, so there are at most $\mathbf{N} - 1$ iterations (remember we start with an identified duck). Therefore, the overall running time is $O(\mathbf{N}^2 \times \mathbf{S} \times (\mathbf{S} + \mathbf{M}))$. With all those variables being bounded by $50$, that should fit in time.

## Test Set 2

In Test Set 2, we need to speed up things significantly. We can start by making use of a more refined version of our consistency observations. As you can see in the proof, we do not need to require every pair of statements or a statement and a meeting to be consistent: only those that are "consecutive".

Formally, let us call two statements $s_1$ and $s_2$ consecutive in $S$ if they refer to times $t_1$ and $t_2$, respectively, and to bird $b$, and there is no other statement in $S$ that refers to a time $t_3$ such that $t_1 < t_3 < t_2$ and to bird $b$. Similarly, let us call a statement $s$ in $S$ that refers to time $t_1$ and a

meeting $m$ at time $t_2$ consecutive if there is no other meeting at time $t_3$ such that $t_1 < t_3 < t_2$. Notice that consecutive is not a total order, because of statements referring to the same time. However, making it a total order by breaking ties arbitrarily maintains the validity of the theorem.

Then, we can say that a set $S$ of statements and/or meetings with known ducks is consistent if and only if any consecutive pair of them is consistent. The proof is the same as the one given above.

With the observation above, if we can maintain a sorted list of meetings and goose-made statements about each bird, we can check the consistency for each statement $s$ in logarithmic time by only checking its consecutive neighbors (at most $2$ meetings and $2$ statements per bird in $s$). This would already reduce the $O(\mathbf{S} \times (\mathbf{S} + \mathbf{M}))$ inner-most check in the Test Set 1 solution to logarithmic time, a significant improvement.

To maintain that, we can keep a tree structure that allows insertion and lookup in logarithmic time for each bird (like `set` in C++ or `TreeSet` in Java). In this way, every time we process a statement, we simply add the new information to the appropriate birds.

We can further improve by not resetting every time we find a duck. If our structure also allows removal in logarithmic time (like the examples above), we can do the following: When we discover a duck, delete all information coming from their statements. For that, we keep a list for each bird of all information they contributed. Since each piece of information is removed at most once, the overall number of removals is at most the overall number of insertions and does not affect the time complexity.

At this point we have reduced the complexity to $O(\mathbf{N} \times (\mathbf{S} + \mathbf{M}) \times F)$ where $F$ is only logarithmic factors. The $\mathbf{N}$ comes from the external "first duck" iteration. To improve upon that, we can first notice that, if we start from the empty set of ducks and still find some, those birds must always be ducks, and would be ducks when starting from any set. Therefore, in that case, that set of ducks is the answer. If there are no forced ducks, we need to do something different, but we know there is no inconsistency between statements.

At this point, we only need to check consistency between statements and meetings. A statement being inconsistent with a meeting is equivalent to a bird $b_1$ saying that bird $b_2$ is not a duck. Therefore, if $b_2$ were a duck, so would $b_1$. We can represent the situation with a directed graph in which the edges represent these implications. If there is a path in that graph from $b_1$ to $b_2$, then there is an implication (possibly not coming directly from a single statement) that if $b_1$ is a duck, then so is $b_2$. So, if a bird is a duck, then so is every other bird in its strongly connected component (SCC). That means we want to choose an SCC that is as small as possible. Moreover, we want to choose a final component, that is, one that is not pointed to by other components. Non-final components imply other components also being made of ducks, and ultimately bringing in at least one final component. In summary, in this final case the answer is the size of the smallest final SCC, which we can find in linear time.

**Code Jam 2022 - World Finals**

# Analysis: SchrÃ¶dinger and Pavlov

The difficulty of this problem resides on the fact that the state of a box can affect outcomes long after the dog passed them by blocking or not blocking a tunnel. We deal with that difficulty by remembering the state of boxes. Of course, there are way too many boxes to remember the entire state, so we compress it to a smaller amount of information that is manageable. How we do that is different for each test set.

## Test Set 1

In Test Set 1, all tunnels go to nearby boxes. That means once the dog is at box number $i$, the state of boxes with numbers lower than $i - 5$ cannot affect the outcome anymore. So, we can do a dynamic programming solution that computes the probability that there is a cat at box $i$ given the state of the $k$ closest boxes. That is only $O(\mathbf{N} \times 2^k)$ states, and because $k$ is bounded by only $10$ for Test Set 1, that is small enough to solve the problem.

## Test Set 2

For Test Set 2, the number of close boxes that we may need to remember is not bounded by a small number, so we cannot use a solution exponential in it. Let us consider a directed graph with boxes as the nodes and tunnels as directed edges. Because it has the same number of nodes and edges, it is a functional graph. Functional graphs look like forests except they have cycles as the "roots". Notice that only the connected component of the underlying undirected graph that contains the last box matters for the final answer (boxes in other components do not affect whether or not there is a cat in the last box). So, we can discard all other components and assume moving forward the graph is connected, so it's actually a functional graph with a single cycle.

As a thought exercise, let us assume the tunnels graph is actually a directed tree instead (one tunnel is removed). In this case, we can solve the problem by simulating the dog run and remembering things in a smart way. We maintain a forest of the nodes corresponding to every box and the tunnels that may have been used so far, that is, tunnels coming out of boxes that the dog already passed. For each node, we compute the probability that a cat is there. The probabilities of a cat being in any two boxes are independent if they are on different trees of this forest, but otherwise they might not be.

We start with the forest with all nodes and no edges. Probabilities for each node start at $0$, $1/2$c or $1$ depending on whether the box is empty, unknown, or contains a cat, respectively. Then, when we simulate the dog passing through box $i$, that adds one edge to the forest, which merges two trees. Because the probability of the roots of those trees are independent up to this step, we can calculate the probability of the tunnel being used by multiplying the probability that there is a cat in box $i$ and no cat in the destination box. Then, we update all probabilities as the weigthed average of the outcome of both cases (a cat running through the new tunnel or not).

To make the approach above work for an actual functional graph instead of a tree, we need to deal with the sole cycle. We can do this by branching out when we add the first edge of the cycle to the forest (that is, when the dog runs through the box with the smallest number from among those in the cycle). Instead of merging those two components, we consider all $4$ cases for the state of the two boxes at the endpoints of the tunnel. For each one, we can compute its probability with a multiplication as before, because at this points the two endpoint probabilities are independent. Then, instead of merging the two components, we start $4$ computations, one for each case, but in all of them the components are kept separated. At the end, we have $4$

results for the last box. The final result is the weighted average of that box where the weights are the probabilities of each case that we calculated when forking the computation.

**Code Jam 2022 - World Finals**

# Analysis: Slide Parade

In this problem, we are given a simple graph $G$. We are required to find a sufficiently-small circuit that passes through each edge at least once, and passes through all vertices the same number of times.

Let the input graph be $G = (V, E)$.
$V = \{1, 2, \ldots, B\}$, $E = \{(X_1, Y_1), (X_2, Y_2), \ldots, (X_S, Y_S)\}$.

If the circuit exists, let $k$ be the number of times that each vertex is passed through. Collecting the edges in the circuit forms a multigraph $G' = (V, E')$, where $E'$ is a multiset. $G'$ has the following properties:

- The underlying set of $E'$ is $E$. Which means
  - Each edge in $E$ appears at least once in $E'$.
  - Each edge in $E'$ appears in $E$.
- For each node $v$, $indegree(v) = outdegree(v) = k$.

On the other hand, if we could find a multigraph $G'$ satisfying the above properties for some $k$, then we could compute an answer from $G'$ with an Eulerian circuit construction algorithm. An Eulerian circuit is a circuit that visits each edge in a graph exactly once. In the following solutions, we will focus on finding a $G'$ via different approaches.

Let $L$ be the output limit $10^6$. Note that $L$ is also a significant factor while analyzing the complexity.

## $O(L\mathbf{BS})$ approach

In this approach, we will use a max-flow algorithm to find a set of valid multiplicities $d_i$ for each $(X_i, Y_i)$ in $E$. i.e. $(X_i, Y_i)$ appears $d_i$ times in $E'$.

Note that the total number of edges in the cycle would be $k\mathbf{B}$, so the upper bound of $k$ is $\frac{L}{\mathbf{B}}$.
Enumerating all possible $d_i$ is not feasible, but we can try all possible values of $k$, and see if a set of valid $d_i$ can be generated under the fixed $k$. Once we find a set of valid $d_i$, then $G'$ is also determined.

This can be reduced to a maximum flow problem. We will construct a flow network with source $s$ and sink $t$ to help us find a valid set of $d_i$.

The vertices of the flow network are:

- The source $s$.
- The sink $t$.
- A node $in_v$ for each $v$ in $V$.
- A node $out_v$ for each $v$ in $V$.

There are $(2\mathbf{B} + 2)$ vertices in the network in total.

The edges of the flow network are:

- $(s, out_v)$ for each $v$ in $V$, with capacity $k$.
- $(out_u, in_v)$ for each $(u, v)$ in $E$. No capacity limit, but the lower bound of flow is 1.

- $(in_v, t)$ for each $v$ in $V$, with capacity $k$.

There are $(\mathbf{S} + 2\mathbf{B})$ edges in the network in total.

The amount of flow through $in_v$ ($out_v$, *resp.*) indicates the indegree (outdegree, *resp.*) of vertex $v$ in $G'$. The amount of flow on $(out_u, in_v)$ indicates the multiplicity of edge $(u, v)$.

Now we search for the maximum flow on the graph. If the total flow is $k\mathbf{B}$ (the maximum possible), then the amount of flow through $in_v$ and $out_v$ is $k$ for each $v$, and we have found a valid set of multiplicities $d_i$. We can then construct $G' = (V, E')$ with those multiplicities, find an Eulerian circuit in $G'$, and output that circuit.

On the other hand, if we don't find such a flow for any possible $k$, then we output IMPOSSIBLE.

Now let's analyze the time complexity of this solution. In this solution, we run a maximum flow algorithm for each $k$. There are $\frac{L}{\mathbf{B}}$ possibilities of $k$, and the graph has at most $O(\mathbf{B})$ vertices and $O(\mathbf{S})$ edges. Thus the total time complexity is $O(L\mathbf{BS})$, assuming Dinic's $O(\mathbf{B^2S})$ algorithm is chosen for solving maximum-flow. This can be made faster by computing a maximum flow for each $k$ by starting with the flow found for the previous value of $k$.

## $O(\mathbf{S^2})$ approach

The previous algorithm could be somewhat slow. It would be good if we could generate a graph $G'$ more directly. Consider this simple iterative algorithm that produces a $G'$:

- Choose an edge (u, v) that is not yet in $G'$.
- Add edges on a path from building 1 to building u to $G'$.
- Add the edge (u, v) to $G'$.
- Add edges on a path from building v to building 1 to $G'$.
- Repeat until each edge occurs at least once in $G'$.

The $G'$ produced by this algorithm is able to produce a circuit, since we can simply follow the edges in the same order they were added. But the buildings would not necessarily be visited an equal number of times. So, it would be good if we could do the following instead:

- Choose an edge $(u, v)$ that is not yet in $G'$.
- Find a set $A$ of edges, such that the indegree and outdegree of each node is equal in $A$, and $A$ includes $(u, v)$.
- Add $A$ to $G'$.
- Repeat until each edge occurs at least once in $G'$.

If this succeeds, it would yield a $G'$ with all the required properties we listed earlier. Also, if every set $A$ found in the algorithm was as small as possible, that is, if the indegree and outdegree of each node in each $A$ was $1$, then the total number of edges in $G'$ would be at most $SB$, which is at most $10^6$, which conveniently is the limit in the problem!

Now, the question arises — if the problem is possible, can we always find such a set $A$ for any edge $(u, v)$? We can show that we can.

The problem of finding a set $A$ of edges can be reduced to finding a perfect bipartite matching, on a graph similar to the flow graph in the previous solution.

The bipartite graph consists of vertices $(s_1, s_2, \ldots s_{\mathbf{B}})$ and $(t_1, t_2, \ldots t_{\mathbf{B}})$. There is an edge $(s_u, t_v)$ if and only if $(u, v)$ is in $G$. If we use an edge in the perfect matching, then we add the corresponding edge in $G$ to $A$.

We need to show that a perfect matching exists in this graph if the problem instance is possible. To do that, we apply Hall's marriage theorem, which is a common technique for proving whether a graph has a perfect matching.

What we need to prove to use the theorem is the following: if there is a solution to the problem, then for any subset $S$ of the nodes$(s_1, s_2, \ldots s_{\mathbf{B}})$, let $T$ be the set of all nodes adjacent to a node in $S$. Then $|S| \leq |T|$. (We must also show that a similar result holds if we start with a subset of the nodes $(t_1, t_2, \ldots t_{\mathbf{B}})$, but the proof for that is the same.)

Assume there is a solution to the problem, which has a corresponding multigraph $G'$, in which the indegree and outdegree of each node is $k$.

For each $i$, let $g(s_i)$ be the number of edges in $G'$ whose tail is node $i$.

For each $i$, let $g(t_i)$ be the number of edges in $G'$ whose head is node $i$.

Now for any pair of $S$ and $T$ above, $\sum_{s \in S} g(s) \leq \sum_{t \in T} g(t)$ since the edges in $G'$ whose head is in $T$ must include all the edges in $G'$ whose tail is in $S$, plus possibly some more. But because $G'$ corresponds to a solution, the values of $g$ must all be equal to $k$. So we can conclude that $|S| \leq |T|$ as required.

But this is not exactly the bipartite matching problem we need to solve - we need to include some fixed edge $(s_u, t_v)$ in the matching each time. So remove all other edges adjacent to $s_u$ or $t_v$ from the bipartite graph, and now define $g$ as follows:

For each $i$, let $g(s_i)$ be the number of edges in $G'$ whose tail is node $i$, excluding those edges deleted from the bipartite graph.

For each $i$, let $g(t_i)$ be the number of edges in $G'$ whose head is node $i$, excluding those edges deleted from the bipartite graph.

Consider a set $S$ which does not contain $s_u$.

$k|S| - k < \sum_{s \in S} g(s)$, since less than $k$ edges were removed from the graph. Also, $\sum_{t \in T} g(t) \leq k|T|$ since $k$ is still the maximum value of any $g(t)$. We still have $\sum_{s \in S} g(s) \leq \sum_{t \in T} g(t)$ as before, so

$k|S| - k < \sum_{s \in S} g(s) \leq \sum_{t \in T} g(t) \leq k|T|$

$\therefore k|S| - k < k|T|$

$\therefore |S| - 1 < |T|$

$\therefore |S| \leq |T|$ as required.

The same result holds for sets $S$ which do contain $s_u$, since we simply match $s_u$ with $t_v$ and are left with a set $S$ without $s_u$ and we can proceed as above.

If we fail to find a perfect matching for any edge $(s_u, t_v)$, then we can deduce that it's impossible to find a solution to the problem.

This application of Hall's marriage theorem is also known as Birkhoff's theorem on doubly stochastic matrices.

In the current approach, we find a perfect matching on $\mathbf{S}$ different bipartite graphs. Each of them has $O(\mathbf{B})$ vertices and $O(\mathbf{S})$ edges. The total time complexity is $O(\mathbf{BS^2})$ if we use a flow-based algorithm.

However, this approach can still be optimized further. We can make use of a previous result to avoid re-calculating the whole matching every time.

We first find an arbitrary perfect matching as the base matching. Then, for each edge $(s_u, t_v)$, if it's not in the base matching, we remove the edges from the matching that were adjacent to $s_u$ and $t_v$, and add the edge $(s_u, t_v)$. This would make exactly two vertices unmatched (those who originally matched with $s_u$ and $t_v$ in the base matching), and we could find the new matching by searching for an augmenting path between the two unmatched vertices.

Finding the base matching takes $O(\mathbf{BS})$ time, and for each edge $(s_u, t_v)$, it takes $O(\mathbf{S})$ time to find an augmenting path. There are $\mathbf{S}$ edges in the bipartite graph, so the total time complexity is $O(S^2)$.

**Code Jam 2022 - World Finals**

# Analysis: Triangles

## Test Set 1

In Test Set 1 there are so few points that we can just try every possible way to assign them to triangles. With a maximum of $\mathbf{N} = 12$ points, there are $12!/(3!^4 \cdot 4!) = 15400$ ways of getting $3$ or $4$ triangles, even less to get $1$ or $2$, and a single way to get $0$, which means there are less than $4 \cdot 15400 + 1 = 61601$ things to try. For each one, we need to check that every triangle is valid (i.e., made up of non-collinear points) and that each pair of triangles meets the definition given in the statement. Since there are very few triangles, this requires quite a bit of code but not a lot of computation time.

## Test Set 2

We solve this problem by constructively proving the following theorem: a set $S$ of $3t$ points can be split to form $t$ triangles that fulfill the conditions of the statement if and only if it does not contain a subset of $2t + 1$ collinear points.

The case where $t = 1$ is trivial. If $t > 1$ we consider $3$ different cases. Let $C$ be the largest subset of $S$ made up of collinear points.

- If $|C| < 2t - 1$, we find a triangle that is separated from all other points by a line and then recursively solve an instance with less points. Since a triangle can use at most $2$ points from $C$, this does not make the remaining set exceed the maximum number of collinear points allowed. One way of finding such a triangle is to get the $2$ maximal points in the lexicographical order (i.e., ordering by X-coordinate and breaking ties by Y-coordinate) $v$ and $w$, and then find the third points $x_1$ and $x_2$ that minimize and maximize, respectively, the angle $vwx_i$, breaking ties by the distance $|wx_i|$. Then, pick $x_1$ if the angle is less than $\pi$ or $x_2$ otherwise (in that case, $vwx_2$ is guaranteed to be greater than $\pi$). Notice that this makes The line $wx_i$ separate the three points from all others (there could be more points over that line, but $w$ and $x_i$ are the extreme points over it, so any triangles we can make from the remaining points will not interfere with this triangle).
- If $|C| \geq 2t - 1$ and $|C| \neq 3$, let $B, D \subseteq S$ be the subsets of points on each side of the line that goes through $C$. If neither is empty, assuming without loss of generality that $|B| \leq |D|$, we can match the lexicographically greatest $2|B|$ points of $C$, let us call that $C'$ with $B$ to recursively solve $B \cup C'$ and $D \cup (C \setminus C_1)$. Notice that there is a separation line between those two sets, so the recursive solutions do not interfere with each other. If one of $B$ or $C$ is empty we can take the two lexicographically greatest points in $C$ and match it with one of the remaining points, as in the previous step, solving the rest recursively. If $|C| = 2$, this makes a single triangle. Otherwise, $|C| > 3$, so after removing two points from $C$ and one point from $S \setminus C$, the requirements of the theorem applies to the remaining points.
- Otherwise, $|C| = 3$ and $t = 2$. If the convex hull of $S$ has $3$ vertices, we can use those for one triangle, and the other $3$ for the other. If the convex hull of $S$ has $5$ or $6$ vertices, it contains at least two from $C$, so we can use those $2$ plus a single intermediate or adjacent point to form one triangle, and the rest for the other. If the convex hull of $C$ has $4$ points, there are a few cases to consider depending on where the other $2$ points are located, but all are solvable. Implementation-wise, we can simply try all possible ways to match it, since there is a small amount anyway.

Because of the above, we can do the following: find a largest set of collinear points in the input $C$. If it is larger than $\lceil 2\mathbf{N}/3 \rceil$, ignore points from it to make it equal to that amount. After that, ignore points not in $C$ to make the set of non-ignored points have size multiple of $3$. We will match all those points into triangles using the above idea. If we keep the points sorted lexicographically, we can implement each step finding a new triangle in linear time, except for the fact that after each step of the first type we may need to recalculate $C$.

To speed up the calculation of largest subset of collinear points, we can use three techniques. The first two speed it up in complexity, but can be really slow in practice due to a combination of large constants and an accumulation of logarithmic factors depending on the exact implementation.

- Calculate all subsets of collinear points and keep them updated and on a priority queue to quickly identify the largest. There are $\mathbf{N}$ point removals and each one updates up to $\mathbf{N}-1$ sets, so there are $O(\mathbf{N}^2)$ updates overall. If the sets are linked lists and the priority queue is over an array (the sizes are limited to the range $1$ through $\mathbf{N}$), this can be theoretically be done in $O(\mathbf{N}^2)$ overall. However, since an initial calculation of $C$ for the entire input necessitates $O(\mathbf{N}^2 \log \mathbf{N})$ operations, it might be tempting to use less efficient but quicker to code structures.
- If $|C|$ is a lot smaller than the threshold needed to not be in the first case, we can simply take multiple steps of the first case without recalculating. It can be shown that doing this leads to only a logarithmic number of recalculations. Notice that this still makes the overall complexity $O(\mathbf{N}^2 \log^2 \mathbf{N})$ and it has large constants because the input size before each recalculation is not halved, but reduced by about $1/5$ in the worst case.

A simpler and a lot faster way is to not calculate $C$ explicitly at all. Just keep doing the first case until you detect all remaining points are collinear (this happens when both $vwx_1$ and $vwx_2$ are planar angles). At that point, undo just enough of the latest made triangles to make that identified set of collinear points big enough to not be in the first case, and continue with the second and third cases. This makes the overall time complexity $O(\mathbf{N}^2)$ and, not having complicated structures or logarithms with small bases, it does not hide any large constants.

**Code Jam 2022 - World Finals**

# Analysis: Wonderland Chase

## Test Set 1

The labyrinths of Wonderland can be viewed as a undirected graph. If Alice can get to a cycle before the Queen can reach her, then Alice will be safe. This is because Alice can always pick a direction of travel away from the Queen around the cycle. Conversely, if Alice cannot get to a cycle before the Queen, then the Queen will catch her after at most $2 \times \mathbf{J}$ moves. Every second move, Alice may move to a node, so after $2 \times \mathbf{J}$ moves Alice must have entered a cycle because otherwise the Queen would have caught Alice.

Using these observations, we can formulate a dynamic programming solution. Define a recursive function `solution(alice, queen, total_moves)` that is true if the Queen can catch Alice in at most `total_moves` with both moving optimally and false otherwise. The solution to the problem is the minimum value for `total_moves` such that `solution(`$\mathbf{A}$`, `$\mathbf{Q}$`, total_moves)` is true. If it is never true, then Alice is safe.

We can compute `solution(alice, queen, total_moves)` using a recurrence relation in which we try all moves for the Queen and all moves for Alice in two nested loops calling `solution` recursively. The Queen is always trying to make the answer true, and Alice is always tryng to make it false.

An upper bound on the complexity of our solution is $O(\mathbf{J}^5)$, since there are $O(\mathbf{J}^3)$ states and computing a state requires at most $O(\mathbf{J}^2)$ time. A better complexity analysis is possible, but we already know this will be fast enough to pass Test Set 1.

## Test Set 2

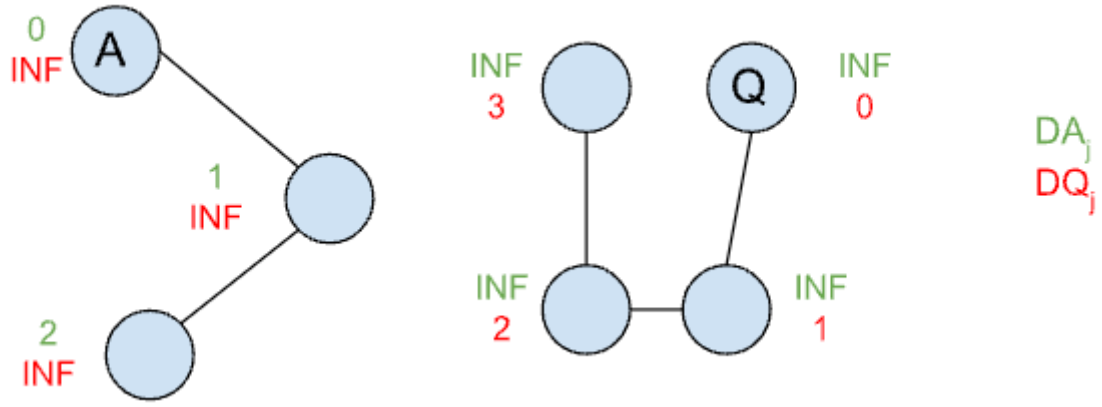
A faster solution is needed for Test Set 2. We will need some more observations. Call a node *good* when, if Alice reaches it before being caught, Alice will always be safe. That is, if Alice reaches a good node, then regardless of where the Queen is, Alice can always move in a way that is safe. Let's consider an algorithm for finding all good nodes.

Assuming a connected graph, leaves (nodes with degree 1) are never good because Alice can become cornered in them. We can start by deleting them. In fact, if we iteratively remove leaves until there are none left, then all the remaining nodes must be good, since Alice can never be cornered. This algorithm can be implemented in linear time using a queue of leaves and keeping track of the degree of each node throughout deletions.

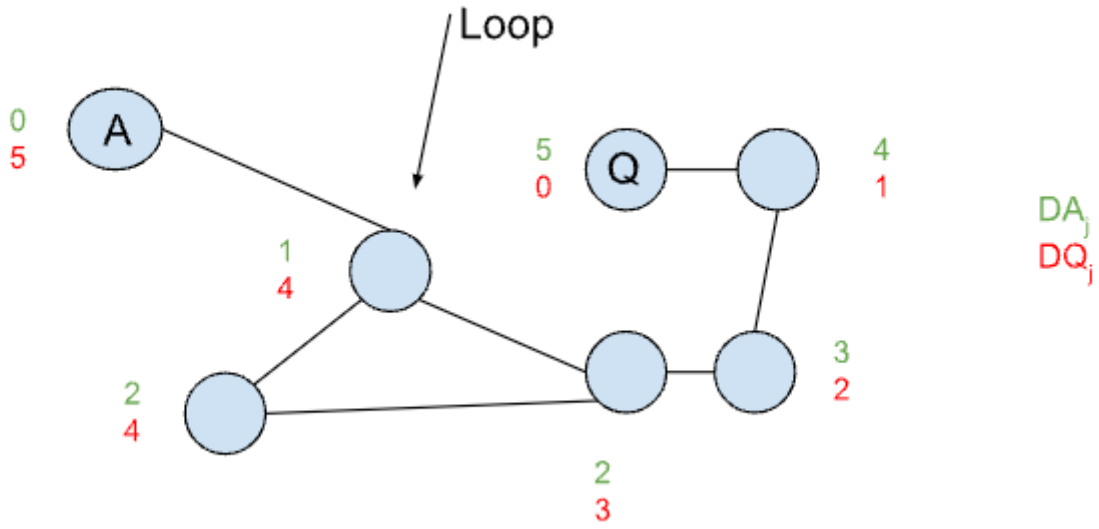
Define $\mathbf{DA}_u$ as the shortest path from Alice's starting node to a node $u$. Likewise, define $\mathbf{DQ}_u$ as the shortest path from the Queen. For a node $j$, if $\mathbf{DA}_j < \mathbf{DQ}_j$, then Alice can safely reach that node before being caught. We can prove this using a contradiction: if the Queen was able to intercept Alice anywhere on her path, then the Queen must have reached some node on Alice's shortest path before Alice, which would imply that the Queen can reach $u$ first. Note that $\mathbf{DA}$ and $\mathbf{DQ}$ can be computed by running one Breadth-First Search (BFS) each and storing the resulting table of distances.

We can use our observations to solve the problem by considering a few cases. Alice will be safe if:

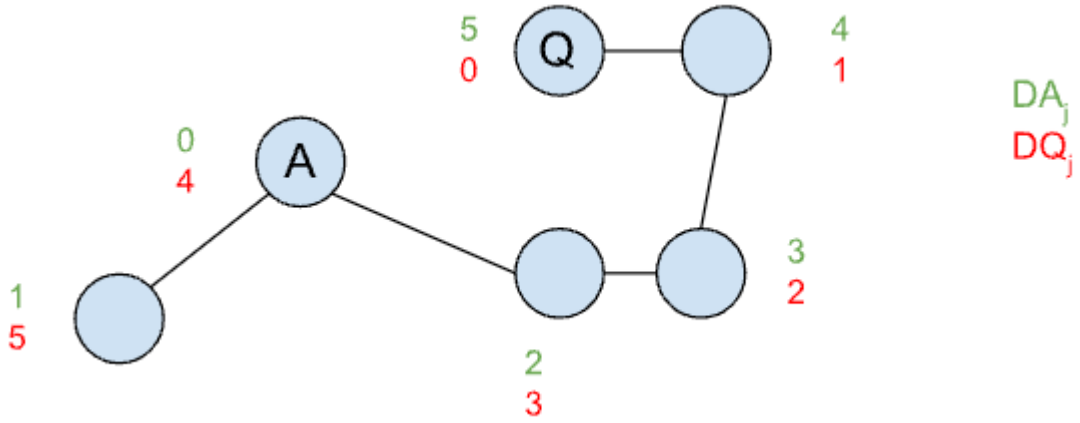
- The graph is disconnected, meaning Alice and the queen start in two disconnected components. This can be checked by looking at either $\mathbf{DA}$ or $\mathbf{DQ}$.




- Alice can enter a good node $j$ such that $\mathbf{DA}_j < \mathbf{DQ}_j$.




Otherwise, Alice will get caught. Since Alice's strategy is to maximize the number of moves until she is caught, she will pick the junction that has the maximum distance from the Queen with the condition that she can get to it first ($\mathbf{DA}_j < \mathbf{DQ}_j$).




Since the solution requires only three linear passes over the graph (one to find good nodes, and two BFS runs), the total complexity is $O(\mathbf{J} + \mathbf{C})$.